บทที่ 3

คอมโพเนนต์พื้นฐานและการจัดการปฏิสัมพันธ์กับผู้ใช้

ในสภาพแวดล้อมการพัฒนาแอปพลิเคชันด้วย React Native คอมโพเนนต์พื้นฐาน (Core Components) ทำหน้าที่เป็นตัวเชื่อมต่อ (Bridge) ไปยังมุมมองดั้งเดิม (Native UI Views) ของระบบปฏิบัติการ iOS (เช่น **UIView**, **UIImageView**, **UITextField**) และ Android (เช่น **View**, **ImageView**, **EditText**) (React Native, 2024) ส่งผลให้แอปพลิเคชันสามารถประมวลผลอินเทอร์เฟซด้วยประสิทธิภาพระดับเนทีฟอย่างแท้จริง

3.1 คอมโพเนนต์แสดงผลรูปภาพ Image (Local และ Network Assets)

คอมโพเนนต์ **Image** ใช้สำหรับการแสดงผลรูปภาพในแอปพลิเคชัน (React Native, 2024) ทั้งภาพประเภทสเตติกภายในเครื่อง (Static Local Assets) และภาพที่ดาวน์โหลดผ่านอินเทอร์เน็ต (Network Images)

1. การแสดงผลรูปภาพจาก Local Assets

เป็นการดึงภาพที่เก็บรวมอยู่ในโฟลเดอร์ของโปรเจกต์ โดยเรียกใช้ฟังก์ชัน **require()** ในการอ้างอิงตำแหน่งไฟล์ ซึ่งระบบจะประมวลผลขนาดของรูปภาพให้โดยอัตโนมัติตอนทำการบิลด์ (Build Time):

```
import { Image, StyleSheet, View } from 'react-native';

export default function LocalImageExample() {
  return (
    <View style={styles.container}>
      <Image
        source={require('../../assets/images/react-logo.png')}
        style={styles.localImage}
   />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    justifyContent: 'center',
    alignItems: 'center',
  },
  localImage: {
    width: 100,
    height: 100,
  },
```

```
});
```

2. การแสดงผลรูปภาพจากเครือข่ายอินเทอร์เน็ต (Network Images)

เป็นการโหลดรูปภาพผ่านโปรโตคอล HTTP/HTTPS โดยกำหนดค่า `source` เป็นออบเจกต์ที่มีพร็อพส์ `uri`:

ข้อควรจำ (Important): กฎเหล็กของ Network Images: หากเป็นการโหลดภาพผ่าน `uri` ผู้พัฒนา **จำเป็นต้องระบุความกว้าง (width) และความสูง (height) ในสไตล์เสมอ** มิฉะนั้นคอมโพเนนต์จะไม่ทราบขนาดพื้นที่จำลองและไม่แสดงผลภาพขึ้นมา (ขนาดเริ่มต้นจะเป็น 0x0 พิกเซล)

```
<Image
  source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
  style={{ width: 50, height: 50 }}
/>
```

3. การปรับขนาดสัดส่วนด้วย resizeMode

คุณสมบัติ `resizeMode` ทำหน้าที่กำหนดทิศทางว่ารูปภาพจะจัดสัดส่วนอย่างไรเมื่อกรอบของรูปภาพ (สไตล์กว้าง/สูง) มีขนาดต่างจากภาพจริง:

• `cover` (ค่าเริ่มต้น): ยืด/ขยายรูปภาพให้ครอบคลุมเต็มพื้นที่กรอบ โดยรักษาอัตราส่วนภาพไว้ (ทำให้บางส่วนของภาพอาจถูกตัดออก)

• `contain`: ยืด/หดรูปภาพให้อยู่ภายในพื้นที่กรอบได้ครบถ้วนทั้งหมด โดยรักษาอัตราส่วนภาพไว้ (อาจเกิดพื้นที่ว่างซ้าย-ขวา หรือบน-ล่าง)

• `stretch`: ยืดขยายรูปภาพให้เต็มกรอบโดยตรง โดยละทิ้งการรักษาอัตราส่วนเดิม (ภาพจะบิดเบี้ยวได้)

• `center`: จัดกึ่งกลางรูปภาพในเฟรมโดยรักษาขนาดดั้งเดิมไว้ (ไม่มีการยืดขยาย)

3.2 คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด

คอมโพเนนต์ `TextInput` ใช้ในการเปิดพื้นที่รับข้อมูลตัวอักษรหรือข้อความจากผู้ใช้งานผ่านแป้นพิมพ์เสมือน (Virtual Keyboard) (React Native, 2024)

1. พร็อพส์พื้นฐานที่จำเป็นสำหรับการสร้างฟอร์ม

• `value`: ตัวแปรสถานะที่เก็บค่าข้อความปัจจุบัน

• `placeholder`: ข้อความแนะนำแบบจางที่จะแสดงเมื่อไม่มีการพิมพ์ข้อความใด ๆ

• `onChangeText`: เหตุการณ์สัมผัสที่จะเรียกฟังก์ชันกลับคืน (Callback Function) ส่งสตริงค่าล่าสุดกลับมาทุกครั้งที่ปุ่มแป้นพิมพ์ถูกกดพิมพ์

• `secureTextEntry`: กำหนดให้เป็น `true` เพื่อจำลองการกรอกรหัสผ่าน (ตัวอักษรจะเปลี่ยนเป็นสัญลักษณ์จุดวงกลมบดบังสายตา)

```
import { useState } from 'react';
import { TextInput, StyleSheet, View } from 'react-native';

export default function InputExample() {
  const [password, setPassword] = useState('');

  return (
    <View style={styles.container}>
      <TextInput
      placeholder="กรอกรหัสผ่านของคุณ"
        value={password}
        onChangeText={setPassword}
        secureTextEntry={true}
        style={styles.input}
   />
    </View>
);
}

const styles = StyleSheet.create({
  container: {
    padding: 16,
 },
  input: {
    height: 48,
    borderWidth: 1,
    borderColor: '#ccc',
    paddingHorizontal: 12,
    borderRadius: 8,
  },
});
```

2. การควบคุมพฤติกรรมคีย์บอร์ด (Keyboard Control)

ในอุปกรณ์เคลื่อนที่ การจัดเรียงประเภทปุ่มบนแป้นพิมพ์ให้ตรงกับบริบทของอินพุตเป็นสิ่งสำคัญอย่างยิ่งเพื่ออำนวยความสะดวกให้ผู้ใช้งาน:

• `keyboardType`: ระบุรูปแบบคีย์บอร์ดที่ทำงานขึ้นมา

  • `'default'`: คีย์บอร์ดข้อความปกติ

  • `'numeric'`: คีย์บอร์ดปุ่มตัวเลข (เหมาะสำหรับการป้อนยอดเงิน รหัส OTP)

• `'email-address'`: คีย์บอร์ดที่มีปุ่ม @ และ .com ขึ้นมาอำนวยความสะดวก

• `'phone-pad'`: แป้นพิมพ์ปุ่มหมายเลขโทรศัพท์ที่มีเครื่องหมายบวก/ดอกจัน

• `autoCapitalize`: ควบคุมการขึ้นต้นด้วยอักษรตัวใหญ่ภาษาอังกฤษอัตโนมัติ (แนะนำให้ใช้ `'none'` สำหรับอีเมลและรหัสผ่าน)

• `autoCorrect`: เปิด/ปิดระบบเดาคำศัพท์และแก้ไขข้อความอัตโนมัติของตัวระบบปฏิบัติการ (`true` หรือ `false`)

• `returnKeyType`: เปลี่ยนรูปภาพปุ่มตกลง/ทำรายการที่มุมขวาด้านล่างของคีย์บอร์ด (เช่น `'done'`, `'next'`, `'search'`, `'go'`)

3.3 การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ View

โดยปกติแล้ว คอมโพเนนต์ `View` พื้นฐานจะไม่มีความสามารถในการตอบสนองคำสั่งเลื่อนหน้าจอ (Scrolling) หากเนื้อหาในหน้าจอนั้นมีความยาวเกินกว่าพื้นที่ขอบเขตจอแสดงผลจริง เนื้อหาด้านล่างจะถูกตัดทิ้งและไม่สามารถเลื่อนอ่านได้ (React Native, 2024)

1. พฤติกรรมของ ScrollView

`ScrollView` คือตู้คอนเทนเนอร์แบบเลื่อนดูเนื้อหาภายในได้ โดยจะทำการแสดงผลแถบแถบเลื่อน (Scrollbar) อัตโนมัติเมื่อเนื้อหาภายในขยายความยาวเกินกว่าขอบเขตหน้าจอ

```
import { ScrollView, Text, StyleSheet } from 'react-native';

export default function LongContent() {
  return (
    <ScrollView style={styles.scroll}>
      <Text style={styles.text}>ย่อหน้าที่ 1...</Text>
      <Text style={styles.text}>ย่อหน้าที่ 2...</Text>
      {/* เพิ่มเนื้อหาจำลองที่มีความยาวลงไปได้เรื่อย ๆ */}
    </ScrollView>
 );
}

const styles = StyleSheet.create({
  scroll: {
    flex: 1,
    backgroundColor: '#fff',
  },
  text: {
    fontSize: 18,
    marginVertical: 40,
  },
});
```

2. การเปรียบเทียบ View กับ ScrollView

ตารางที่ 3.1 การเปรียบเทียบคุณสมบัติระหว่าง View และ ScrollView

คุณสมบัติ	View	ScrollView
การเลื่อนหน้าจอ (Scrolling)	ไม่ทำตามการปัดสัมผัสเลื่อนหน้าจอ	รองรับการเลื่อนแนวตั้งและแนวนอน
พื้นที่ยืดแกนเลย์เอาต์	จัดตำแหน่ง Layout ด้วย Flexbox ได้ทันที	จัดเลย์เอาต์เฉพาะจุดผ่าน contentContainerStyle
การเรนเดอร์หน่วยความจำ	รวดเร็ว แสดงเฉพาะของภายในกล่อง	เรนเดอร์คอมโพเนนต์ลูกทั้งหมดในครั้งเดียวตอนโหลด
กรณีการใช้งานที่เหมาะสม	โครงสร้างบล็อกเลย์เอาต์, ป้าย, พื้นที่ที่ไม่ล้นจอ	หน้าจอกรอกข้อมูลแบบฟอร์ม, หน้าเนื้อหายาว ๆ เช่น ข้อตกลง

คำเตือน (Warning): การใช้สไตล์กับ ScrollView: หากคุณใช้ `style` บน `ScrollView` จะเป็นการจัดลักษณะทางกายภาพภายนอกของตัวเลื่อนเอง แต่หากคุณต้องการกำหนดช่องไฟ ระยะห่าง (Padding) หรือการจัดกึ่งกลางเนื้อหาลูกที่อยู่ข้างในพื้นที่เลื่อน **คุณต้องใช้งานผ่านพร็อพส์ `contentContainerStyle` เท่านั้น**

3.4 การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable

ในยุคแรกเริ่ม React Native ใช้กลุ่มคอมโพเนนต์ตระกูล Touchable (เช่น `TouchableOpacity`, `TouchableHighlight`) ในการสร้างปุ่ม แต่ในปัจจุบันเวอร์ชันใหม่ ๆ ได้นำเสนอคอมโพเนนต์ที่ยืดหยุ่นและก้าวหน้ากว่าคือ `Pressable` (React Native, 2024) ซึ่งรองรับการตรวจจับระดับการกดสัมผัสในสภาพการใช้งานที่ซับซ้อนขึ้น

1. เหตุการณ์สัมผัสต่าง ๆ บน Pressable

- `onPress`: สัมผัสปุ่มและปล่อยตัวในเวลาปกติ (การคลิกปุ่มทั่วไป)
- `onLongPress`: กดปุ่มสัมผัสแช่ไว้เป็นเวลาอย่างน้อย 500 มิลลิวินาที
- `onPressIn`: สัมผัสเริ่มแรกเมื่อนิ้วแตะโดนปุ่ม
- `onPressOut`: จังหวะที่ยกนิ้วขึ้นจากพื้นที่สัมผัสปุ่ม

2. การจัดการสถานะ Active/Pressed สไตล์ไดนามิก

Pressable ยกระดับความสามารถโดยสามารถรับฟังก์ชันในคุณสมบัติ style หรือ children เพื่อรับทราบข้อมูลสถานะแอปพลิเคชันว่าผู้ใช้งานกำลังเอานิ้วกดคาไว้อยู่หรือไม่ผ่านตัวแปรบูเลียน { pressed }:

```
import { Pressable, Text, StyleSheet } from 'react-native';

export default function CustomButton() {
  return (
    <Pressable
      onPress={() => console.log('Button Pressed!')}
      style={({ pressed }) => [
        styles.button,
        {
     backgroundColor: pressed ? '#1d4ed8' : '#2563eb', // สีเข้มขึ้นเมื่อถูกกด
          transform: [{ scale: pressed ? 0.98 : 1.0 }],    // ย่อตัวลงเล็กน้อยจำลองน้ำหนักกด
        }
   ]}
    >
      <Text style={styles.text}>บันทึกข้อมูล</Text>
    </Pressable>
 );
}

const styles = StyleSheet.create({
  button: {
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 8,
    alignItems: 'center',
  },
  text: {
    color: 'white',
    fontWeight: 'bold',
    fontSize: 16,
  },
});
```

3.5 การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView

อุปกรณ์สมาร์ทโฟนยุคใหม่มีดีไซน์ที่หลากหลาย เช่น หน้าจอไร้ขอบที่มีรอยบาก (Notch) มีกล้องทรงหยดน้ำ หรือแถบนำทางระบบปฏิบัติการอยู่ด้านล่างหน้าจอ หากเขียนวางคอมโพเนนต์ชิดขอบบนหรือขอบล่างเกินไป เนื้อหาอาจจะถูกกล้อง รอยบาก หรือแถบสถานะ (Status Bar) บดบัง

การใช้งาน SafeAreaView

คอมโพเนนต์ **SafeAreaView** จะคำนวณและเว้นพื้นที่ระยะขอบปลอดภัยเฉพาะอุปกรณ์นั้น ๆ ให้อัตโนมัติ (React Native, 2024) (มีผลเด่นชัดบนระบบ iOS) ทำให้ผู้พัฒนาไม่ต้องเขียนสไตล์ขยับความสูงด้วยค่าคงที่ตายตัว:

```
import { SafeAreaView, View, Text, StyleSheet } from 'react-native';

export default function SafeLayout() {
  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.content}>
        <Text>เนื้อหาของแอปพลิเคชันที่ได้รับการปกป้องจากรอยบากหน้าจอ</Text>
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#f8fafc', // ให้พื้นที่ปลอดภัยเป็นสีเดียวกับแอป
  },
  content: {
    flex: 1,
    padding: 16,
  },
});
```

3.6 แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย TypeScript

การพัฒนาแอปพลิเคชันที่มีขนาดใหญ่ขึ้นจำเป็นต้องคำนึงถึงความสะดวกในการบำรุงรักษาโค้ดและการนำชิ้นส่วนอินเตอร์เฟซกลับมาใช้ซ้ำ (Component Reusability) โดยสร้าง **Custom Components** และส่งผ่านค่าข้อมูลความต้องการผ่าน **Props**

1. แนวคิดการแยกสร้าง Custom Component

สมมติว่าเราต้องการสร้างการ์ดแนะนำข้อมูลผู้ใช้งานที่มีรูปโปรไฟล์ ชื่อ และปุ่มติดต่อ หากประกาศไว้ทั้งหมดบนหน้าหลัก โค้ดจะยาวและซ้ำซ้อน เราจึงควรแยกสร้างเป็นไฟล์ย่อยภายนอก

2. การประกาศคุณสมบัติและกำหนดโครงสร้างข้อมูลด้วย TypeScript Interface

ในระดับมืออาชีพการใช้ TypeScript ร่วมกับ React Native จะช่วยควบคุมคุณภาพการพัฒนาได้อย่างดีเยี่ยม โดยจะนำ **interface** มากำหนดเป็นตัวกรองข้อมูลว่าคอมโพเนนต์ย่อยนั้น ๆ ต้องการตัวแปรชนิดใดบ้าง และตัวแปรใดเป็นสิ่งจำเป็นหรือมีตัวเลือกอื่น ๆ:

```
// 1. สร้างไฟล์ย่อยที่ตั้งชื่อว่า components/UserProfileCard.tsx
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

// กำหนดประเภทข้อมูล Props ด้วย TypeScript Interface
export interface UserProfileCardProps {
  name: string;
  avatarUrl: string;
  role: string;
  isActive?: boolean; // เครื่องหมาย ? แสดงว่าเป็นตัวเลือก (Optional) ไม่ส่งมาก็ไม่เกิดข้อผิดพลาด
  onPressDetails?: () => void; // ฟังก์ชัน Callback สำหรับส่งสัญญาณเมื่อกดการ์ด
}

export default function UserProfileCard({
 name,
  avatarUrl,
  role,
  isActive = false, // กำหนดค่าเริ่มต้นให้กับพร็อพส์ที่เป็นตัวเลือก
  onPressDetails,
}: UserProfileCardProps) {
  return (
    <View style={styles.card}>
      <Image source={{ uri: avatarUrl }} style={styles.avatar} />
      <View style={styles.info}>
        <Text style={styles.nameText}>{name}</Text>
        <Text style={styles.roleText}>{role}</Text>
        <Text style={[styles.statusText, { color: isActive ? '#10b981' :
'#ef4444' }]}>
          {isActive ? '● กำลังทำงาน' : 'o พักผ่อน'}
        </Text>
      </View>
      {onPressDetails && (
        <Pressable onPress={onPressDetails} style={styles.button}>
          <Text style={styles.buttonText}>ข้อมูลเพิ่มเติม</Text>
        </Pressable>
   )}
    </View>
 );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    alignItems: 'center',
    padding: 16,
    backgroundColor: '#fff',
```

```
    borderRadius: 12,
    marginVertical: 8,
    shadowColor: '#000',
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 3,
  },
  avatar: {
    width: 60,
    height: 60,
    borderRadius: 30,
  },
  info: {
    flex: 1,
    marginLeft: 16,
  },
  nameText: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  roleText: {
    color: '#64748b',
    fontSize: 14,
  },
  statusText: {
    fontSize: 12,
    marginTop: 4,
    fontWeight: '500',
  },
  button: {
    backgroundColor: '#0f172a',
    paddingVertical: 6,
    paddingHorizontal: 12,
    borderRadius: 6,
  },
  buttonText: {
    color: '#fff',
    fontSize: 12,
  },
});
```

3. การเรียกใช้งานในหน้าจอหลัก (Parent Component)

เราสามารถเรียกนำเข้าและสร้างผู้ใช้หลาย ๆ คนได้โดยส่งข้อมูลที่ต้องการแตกต่างกันผ่านคุณสมบัติ Props:

```
// 2. เรียกใช้ภายในไฟล์หลัก App.tsx หรือ index.tsx
import { View, StyleSheet, Alert } from 'react-native';
import UserProfileCard from './components/UserProfileCard';

export default function HomeScreen() {
  return (
    <View style={styles.container}>
      {/* ส่งข้อมูลครบสมบูรณ์ */}
      <UserProfileCard
        name="สมชาย ใจดี"
        avatarUrl="https://images.unsplash.com/photo-1535713875002-d1d0cf377fde"
        role="ผู้พัฒนาโมบายแอป"
        isActive={true}
        onPressDetails={() => Alert.alert('ข้อมูลเพิ่มเติม', 'สมชายเป็นผู้พัฒนามานานกว่า 5 ปี')}
      />

      {/* ละตัวแปร isActive และ onPressDetails ที่เป็นออปชัน เสริมความยืดหยุ่น */}
      <UserProfileCard
        name="สมศรี รักเรียน"
        avatarUrl="https://images.unsplash.com/photo-1494790108377-be9c29b29330"
        role="นักออกแบบ UX/UI"
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f1f5f9',
    padding: 16,
  },
});
```

สรุปท้ายบทเรียน

คอมโพเนนต์แสดงผลรูปภาพ Image: คอมโพเนนต์ Image (React Native, 2024) มีหน้าที่แสดงผลรูปภาพในโปรเจกต์ ทั้งภาพภายในเครื่องผ่าน require() และภาพจากอินเทอร์เน็ตผ่านพร็อพส์ uri โดยสำหรับการโหลดภาพออนไลน์นั้นมีข้อบังคับสำคัญคือต้องกำหนดความกว้างและ

ความสูงไว้เสมอมิฉะนั้นภาพจะไม่แสดง และสามารถปรับการจัดวางองค์ประกอบด้วย resizeMode เช่น cover, contain, stretch หรือ center เพื่อรักษาสัดส่วนหรือปรับรูปลักษณ์ตามความเหมาะสม

คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด: คอมโพเนนต์ TextInput (React Native, 2024) เป็นเครื่องมือหลักในการสร้างฟอร์มเพื่อรับข้อมูลข้อความจากผู้ใช้งานผ่านคีย์บอร์ด โดยมีพร็อพส์สำคัญ เช่น value, placeholder, secureTextEntry สำหรับการป้อนรหัสผ่าน และ onChangeText สำหรับคอยดึงข้อมูลการพิมพ์ล่าสุด นอกจากนี้เรายังสามารถเลือกรูปแบบคีย์บอร์ดให้เหมาะกับประเภทการรับค่าได้ด้วย keyboardType รวมถึงการปรับค่า autoCapitalize และ returnKeyType เพื่อให้การตอบสนองดีที่สุด

การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ View: แม้ว่า View จะเรนเดอร์ข้อมูลได้รวดเร็วและใช้ในการจัดวางโครงสร้างทั่วไป แต่หากข้อมูลหน้าจอยาวเกินขอบเขตจอภาพก็จะไม่สามารถเลื่อนอ่านได้ ซึ่ง ScrollView จะเข้ามาช่วยแก้ปัญหาและอำนวยความสะดวกในการเลื่อนเนื้อหาแนวตั้งหรือแนวนอน โดยการจัดระยะห่าง (Padding) หรือขอบเขตการจัดตำแหน่งของเนื้อหาภายในจำลองจะต้องใช้งานผ่านพร็อพส์ contentContainerStyle แทนสไตล์ปกติ

การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable: คอมโพเนนต์ Pressable (React Native, 2024) เป็นเครื่องมือยุคใหม่ที่ถูกพัฒนาต่อยอดขึ้นมาเพื่อตรวจจับเหตุการณ์การกดและสัมผัสแบบซับซ้อน เช่น onPress, onLongPress, onPressIn และ onPressOut โดยผู้พัฒนาสามารถเขียนประกาศฟังก์ชันแบบไดนามิกผ่านพารามิเตอร์ของ `{ pressed }` เพื่อสร้างลวดลายและการตอบสนองของปุ่มเมื่อถูกกดสัมผัสในแบบเรียลไทม์ได้อย่างยืดหยุ่น

การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView: SafeAreaView เป็นคอมโพเนนต์สำคัญในการจัดแจงขอบเขตหน้าจอเพื่อช่วยคำนวณและเว้นพื้นที่ว่างในส่วนของรอยบากหน้าจอ แถบกล้อง หรือแถบนำทางระบบปฏิบัติการโดยเฉพาะบนเครื่อง iOS เพื่อให้เนื้อหาหลักของแอปพลิเคชันไม่ถูกรบกวนหรือทับซ้อนและได้รับการจัดระเบียบตามระยะปลอดภัยของมือถือแต่ละรุ่นอย่างเหมาะสม

แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย TypeScript: การพัฒนาแอปพลิเคชันให้สามารถนำส่วนประกอบ UI มารีไซเคิลใหม่ทำได้โดยการแยกชิ้นส่วนไปเขียนเป็น Custom Component และส่งผ่านตัวแปรต่าง ๆ ผ่านระบบ Props โดยมีการใช้ TypeScript Interface ในการควบคุมความปลอดภัยเพื่อกำหนดความต้องการข้อมูลประเภทจำเพาะ เช่น การใส่เครื่องหมาย ? กำหนดตัวแปรแบบเป็นทางเลือก และฟังก์ชัน Callback สำหรับส่งสัญญาณเมื่อเกิดการสัมผัสอินเทอร์เฟซย่อย

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับคอมโพเนนต์พื้นฐาน (Core Components เช่น `Image`, `TextInput`, `ScrollView`, `Pressable`, `SafeAreaView`) และแนวคิดการจัดการ Props ร่วมกับการใช้ TypeScript ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: เมื่อต้องการแสดงผลรูปภาพจากเครือข่ายอินเทอร์เน็ต (Network Image) ด้วยคอมโพเนนต์ `Image` ข้อใดคือรูปแบบการเขียนและข้อกำหนดที่สำคัญที่สุดเพื่อให้รูปภาพแสดงผลขึ้นมา?

ก. `<Image source="https://example.com/pic.png" />` โดยไม่ต้องระบุขนาด

ข. `<Image source={{ uri: 'https://example.com/pic.png' }} />` โดยไม่ต้องระบุขนาด

ค. `<Image source={{ uri: 'https://example.com/pic.png' }} style={{ width: 100, height: 100 }} />`

ง. `<Image localPath="https://example.com/pic.png" style={{ width: 100, height: 100 }} />`

ข้อ 2: ค่า resizeMode รูปแบบใดของคอมโพเนนต์ `Image` ที่ทำหน้าที่หดหรือขยายรูปภาพเพื่อให้อยู่ภายใต้กรอบของคอมโพเนนต์ได้ครบถ้วนทั้งภาพ โดยยังคงรักษาอัตราส่วนภาพเดิมไว้ครบถ้วน?

ก. `cover`

ข. `contain`

ค. `stretch`

ง. `center`

ข้อ 3: พร็อพส์ (Props) ใดของ `TextInput` ที่ใช้ในการกำหนดให้อักขระที่พิมพ์แสดงผลเป็นจุดกลมดำเพื่อป้องกันความปลอดภัยในการกรอกรหัสผ่าน (Password)?

ก. `passwordMask={true}`

ข. `secureTextEntry={true}`

ค. `keyboardType="password"`

ง. `autoCapitalize="none"`

ข้อ 4: เมื่อต้องการรับอินพุตเฉพาะค่าตัวเลข (เช่น ยอดเงิน หรือรหัส OTP) ผู้พัฒนาควรตั้งค่าพร็อพส์ `keyboardType` บน `TextInput` อย่างไร?

ก. `keyboardType="email-address"`

ข. `keyboardType="phone-pad"`

ค. `keyboardType="numeric"`

ง. `keyboardType="default"`

ข้อ 5: ข้อใดอธิบายความแตกต่างในพฤติกรรมการเรนเดอร์เนื้อหาระหว่าง `View` และ `ScrollView` ได้อย่างถูกต้อง?

ก. `View` สามารถเลื่อนขึ้นลงได้อัตโนมัติ แต่ `ScrollView` จะไม่สามารถเลื่อนหน้าจอได้

ข. `View` เหมาะสำหรับการจัดแถวในเลย์เอาต์ ส่วน `ScrollView` จะจัดแนวด้วย Flexbox ได้ไวกว่า

ค. `View` จะซ่อนเนื้อหาที่ล้นขอบจอและไม่มีแถบเลื่อน ส่วน `ScrollView` จะแสดงพื้นที่และช่วยให้ใช้นิ้วปัดเลื่อนอ่านเนื้อหาที่เกินขอบจอได้

ง. `View` ใช้สำหรับแสดงรูปภาพเท่านั้น ส่วน `ScrollView` ใช้แสดงตัวอักษรเท่านั้น

ข้อ 6: หากต้องการระบุสไตล์ของระยะห่างภายใน (Padding) หรือจัดกึ่งกลางให้กับคอมโพเนนต์ย่อยที่อยู่ภายใต้ `ScrollView` จะต้องส่งคำสั่งสไตล์ผ่านพร็อพส์ใดจึงจะทำงานได้ถูกต้อง?

ก. `style`

ข. `contentContainerStyle`

ค. `innerStyle`

ง. `scrollStyle`

ข้อ 7: ข้อใดกล่าวถึงประโยชน์และความสามารถเด่นของคอมโพเนนต์ `Pressable` ที่โดดเด่นกว่าคอมโพเนนต์ในกลุ่ม Touchable รุ่นดั้งเดิมได้ถูกต้องที่สุด?

ก. `Pressable` สามารถแสดงผลภาพเคลื่อนไหว GIF ได้ในตัวเอง

ข. `Pressable` รองรับการปรับเปลี่ยนรูปแบบสไตล์ชีท (Style) และคอมโพเนนต์ลูก (Children) แบบไดนามิกตามสถานะการกดสัมผัสจริงของนิ้วมือผู้ใช้งานผ่านค่า `{ pressed }`

ค. `Pressable` ทำการเชื่อมต่ออินเทอร์เน็ตได้เร็วกว่าคอมโพเนนต์อื่น

ง. `Pressable` ไม่จำเป็นต้องมีพร็อพส์ `onPress` ในการดักจับพฤติกรรมผู้ใช้งาน

ข้อ 8: วัตถุประสงค์หลักของการประยุกต์ใช้คอมโพเนนต์ `SafeAreaView` คือข้อใด?

ก. เพื่อสแกนและล้างไวรัสออกจากตัวเครื่องสมาร์ทโฟนที่เปิดแอป

ข. เพื่อบันทึกข้อมูลอินพุตของผู้ใช้ลงในฐานข้อมูลเครื่องอย่างปลอดภัย

ค. เพื่อหลีกเลี่ยงการแสดงผลของเนื้อหาแอปพลิเคชันไปทับซ้อนกับพื้นที่ทางกายภาพ เช่น รอยบาก กล้อง หรือแถบแสดงสถานะระบบของเครื่อง

ง. เพื่อเร่งความเร็วในการประมวลผลโค้ด JavaScript ให้ไวขึ้น

ข้อ 9: ภายใต้กฎพื้นฐานของไลบรารี React และ React Native ข้อใดอธิบายทิศทางการไหลของข้อมูลผ่านคุณสมบัติ Props ได้ถูกต้อง?

ก. ไหลจากคอมโพเนนต์ลูกย้อนกลับไปแก้ไขค่าในคอมโพเนนต์แม่โดยตรง

ข. ไหลแบบทิศทางเดียว (Unidirectional) จากคอมโพเนนต์แม่ส่งต่อไปยังคอมโพเนนต์ลูก และเป็นข้อมูลแบบอ่านอย่างเดียว (Read-only) สำหรับลูก

ค. ข้อมูลจะไหลเวียนและแก้ไขได้อิสระผ่านทุกคอมโพเนนต์ในโปรเจกต์โดยไม่ต้องประกาศรับ

ง. ไหลเฉพาะจากไฟล์ CSS เข้าสู่เนื้อหาข้อความในแอปพลิเคชันเท่านั้น

ข้อ 10: ในการเขียนแบบมีระเบียบ (Type-safe) การระบุเครื่องหมายคำถามท้ายชื่อตัวแปรใน TypeScript Interface เช่น `avatarUrl?: string;` มีความหมายอย่างไร?

ก. ตัวแปร `avatarUrl` จะต้องส่งค่าเป็นประเภทข้อมูล Boolean เท่านั้น

ข. ตัวแปร `avatarUrl` มีสถานะเป็น Optional Props (ผู้ใช้งานคอมโพเนนต์ย่อยจะส่งค่ามาให้หรือไม่ส่งค่ามาให้ก็ได้โดยไม่เกิดข้อผิดพลาดในการรันไทม์)

ค. ตัวแปร `avatarUrl` ห้ามนำไปใช้งานภายในคอมโพเนนต์เด็ดขาด

ง. ตัวแปร `avatarUrl` จะมีระบบสุ่มค่าข้อความให้อัตโนมัติ

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: พร็อพส์ `source` ไม่รับสตริง URL ตรง ๆ สำหรับรูปภาพจากอินเทอร์เน็ต แต่ต้องประกาศผ่านโครงสร้างออบเจกต์ที่มีพร็อพส์ `uri`

• ข. ผิด: แม้จะระบุเป็นรูปเล่มออบเจกต์ `uri` แล้ว แต่เนื่องจากรูปภาพผ่านเครือข่ายไม่มีตัวตนในเครื่องก่อนบิลด์ แอปพลิเคชันจึงไม่ทราบสัดส่วนขนาด หากไม่เขียนระบุกว้าง-สูง (`style={{ width, height }}`) ขนาดภาพจะถูกเซ็ตเป็น 0x0 ทำให้ไม่ปรากฏรูป

• ค. ถูก: ต้องเขียนระบุรูปเล่มออบเจกต์ `{ uri: 'URL' }` ร่วมกับการประกาศสไตล์ควบคุมขนาดกว้างและสูงอย่างเด่นชัดเพื่อให้ระบบแสดงรูป

• ง. ผิด: ไม่มีพร็อพส์ชื่อ `localPath` ในการดึงรูปภาพของคอมโพเนนต์ `Image`

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `cover` จะดึงขยายรูปภาพให้เต็มกรอบ ซึ่งอาจทำให้อัตราส่วนของภาพยังคงเดิมแต่ส่วนที่เลยกรอบออกไปจะถูกตัดออก (Cropped)

• ข. ถูก: `contain` จะปรับเปลี่ยนขนาดรูปภาพให้อยู่ในขอบเขตกรอบทั้งหมดโดยที่ไม่มีชิ้นส่วนใดถูกบีบเบี้ยวหรือตัดทิ้ง โดยรักษาอัตราส่วนภาพเดิมไว้ร้อยเปอร์เซ็นต์

• ค. ผิด: `stretch` จะยืดสัดส่วนแกนให้พอดีกล่องซึ่งส่งผลให้หน้าตาของรูปภาพเกิดการยืดบิดเบี้ยวได้

• ง. ผิด: `center` จะวางรูปภาพขนาดปกติไว้กึ่งกลางกล่องโดยไม่มีการขยายขนาดรูปภาพ

เฉลย ข้อ 3: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ไม่มีพร็อพส์ชื่อ `passwordMask` ในคอมโพเนนต์ `TextInput`

• ข. ถูก: การตั้งค่า `secureTextEntry={true}` เป็นมาตรฐานของ React Native ในการสั่งให้ช่องรับข้อมูลซ่อนอักขระจริง และเรนเดอร์เป็นสัญลักษณ์จุดวงกลมเพื่อความปลอดภัย

• ค. ผิด: `keyboardType` ใช้เปลี่ยนปุ่มแป้นพิมพ์คีย์บอร์ด ไม่ใช่หน้าตาข้อความข้างในช่องรับค่า

• ง. ผิด: `autoCapitalize` เป็นเพียงคำสั่งปิดตัวพิมพ์ใหญ่ภาษาอังกฤษ ซึ่งไม่มีฟังก์ชันบังขอบเขตข้อความ

เฉลย ข้อ 4: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `email-address` คีย์บอร์ดจะมีปุ่มสัญลักษณ์ตัวอักษรและปุ่ม @ โผล่มาหลัก ๆ

• ข. ผิด: `phone-pad` จะแสดงตัวเลขรวมถึงเครื่องหมายที่ใช้ต่อสายโทรศัพท์ เช่น *, #, + ซึ่งเกินกว่าตัวเลขข้อมูลปกติ

• ค. ถูก: การส่งค่า `keyboardType="numeric"` จะสั่งให้อุปกรณ์ระบบปฏิบัติการเปลี่ยนมาแสดงแป้นพิมพ์ปุ่มหลักเฉพาะตัวเลข 0-9 เพื่อความรวดเร็วและป้องกันไม่ให้ผู้ใช้พิมพ์ตัวอักษรตัวหนังสือที่ผิดรูปแบบเข้ามาได้

• ง. ผิด: `default` จะเรียกคีย์บอร์ดปุ่มกดตัวอักษรข้อความปกติของสมาร์ทโฟนขึ้นมาทำงาน

เฉลย ข้อ 5: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สลับกัน คอมโพเนนต์ `View` ไม่สามารถเลื่อนหน้าจอได้

• **ข. ผิด:** ทั้งสองตัวทำสไตล์จัดสัดส่วนผ่าน Flexbox ได้ไม่ต่างกันในด้านความยากง่าย

• **ค. ถูก:** `View` มีพฤติกรรมแบบคงที่ หากเนื้อหาภายในมีขนาดใหญ่หรือยาวล้นออกไปนอกพื้นที่ของแม่ เนื้อหาที่ล้นจะถูกตัดการแสดงผลทับไปและเลื่อนดูไม่ได้ ส่วน `ScrollView` จะมีความสามารถดักจับการปัดสัมผัสเลื่อนแกนเพื่อขยับวัตถุที่ล้นเข้ามาในหน้าจอได้ทันที

• **ง. ผิด:** ทั้งสองคอมโพเนนต์ใช้จัดวางหรือใส่ประเภทชิ้นงานรูปภาพข้อความข้างในได้ตามกฎปกติ

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การใช้ `style` บน `ScrollView` จะทำงานควบคุมเฉพาะขอบเขตสไตล์ภายนอกของตัวกรอบเลื่อน แต่ไม่ส่งสไตล์จัดแนวลงไปหาคอมโพเนนต์ลูกด้านใน

• **ข. ถูก:** พร็อพส์ `contentContainerStyle` ถูกออกแบบมาเพื่อนำสไตล์การจัดช่องไฟ (เช่น `padding`) หรือการจัดตำแหน่ง (เช่น `justifyContent`, `alignItems`) ส่งเข้าไปควบคุมกล่องห่อหุ้มลูกภายในพื้นที่เลื่อนได้สมบูรณ์

• **ค. และ ง. ผิด:** ไม่มีพร็อพส์ชื่อดังกล่าวใน React Native SDK

เฉลย ข้อ 7: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `Pressable` ไม่เกี่ยวข้องกับการจำลองรันไฟล์ภาพเคลื่อนไหว

• **ข. ถูก:** `Pressable` เป็นคอมโพเนนต์รุ่นใหม่ที่ออกแบบมาให้ตรวจเช็คสัญญาณแรงกดนิ้วได้ยืดหยุ่น โดยสามารถประเมินพารามิเตอร์ `{ pressed }` เพื่อนำไปเปลี่ยนสีปุ่ม ขนาด หรือสไตล์ของวัตถุข้างในได้แบบเรียลไทม์ ทำให้แอปพลิเคชันดูมีชีวิตชีวาและโต้ตอบได้สมบูรณ์

• **ค. ผิด:** ไม่มีความเกี่ยวดองกับการทำงานเน็ตเวิร์กของระบบ

• **ง. ผิด:** ยังคงจำเป็นต้องใช้พร็อพส์ `onPress` ในการทำงานดักจับสัมผัสคลิกปุ่มเช่นเดียวกับปุ่มทั่วไป

เฉลย ข้อ 8: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ไม่มีระบบรักษาความปลอดภัยประเภทกำจัดโปรแกรมที่เป็นภัยต่อตัวเครื่อง

• **ข. ผิด:** การบันทึกเก็บข้อมูลดั้งเดิมจะพึ่งพากลุ่ม Local Storage เช่น AsyncStorage ไม่ใช่หน้าที่ของ View

• **ค. ถูก:** `SafeAreaView` ทำหน้าที่ปรับปรุงระเบียบกล่องเลย์เอาต์โดยการตัดพิกเซลเว้นระยะในจุดที่มีความเสี่ยงต่อการโดนฮาร์ดแวร์ขอบหน้าจอบัง (เช่น ขอบบนที่ติดกล้อง รอยบาก หรือขอบล่างของปุ่มนำทาง Home Indicator) ให้เรียบร้อยโดยไม่ต้องปรับเอง

• **ง. ผิด:** ไม่ได้มีคุณลักษณะในการคอมไพล์โค้ดหรือปรับปรุงประสิทธิภาพ JavaScript Engine

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** คอมโพเนนต์ลูกไม่สามารถเข้าทำรายการหรือแก้ไขตัวแปรต้นฉบับในคอมโพเนนต์แม่ผ่าน Props ได้โดยตรง ยกเว้นแต่ว่าคอมโพเนนต์แม่จะส่งฟังก์ชันสำหรับ Callback ลงไปให้ใช้งาน

• **ข. ถูก:** ตามหลักสถาปัตยกรรม React การส่งข้อมูลผ่าน Props จะมีลักษณะจากบนลงล่าง (จาก Parent สู่ Child) เท่านั้น และตัวคอมโพเนนต์ลูกที่ได้รับ Props ไปจะมีสถานะใช้อ่านอย่างเดียว (Immutable/Read-only) เพื่อไม่ให้สเตตของโปรแกรมสับสนกระจัดกระจาย

• **ค. ผิด:** ต้องระบุตัวแปรที่รับมาในพารามิเตอร์ฟังก์ชันเสมอเพื่อให้ TypeScript และ JavaScript อ้างอิงตัวแปรได้ถูกต้อง

• **ง. ผิด:** ข้อมูล Props ไม่เกี่ยวกับการเปลี่ยนทางสไตล์ชีทแบบดังกล่าว

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ชนิดของตัวแปรในโจทย์ระบุชัดเจนว่าเป็น `string`

• **ข. ถูก:** ใน TypeScript การเขียนต่อท้ายคีย์ด้วยเครื่องหมายคำถาม (`?`) บ่งบอกว่าพร็อพส์นั้น ๆ เป็น "Optional" (ไม่จำเป็นต้องระบุ) ทำให้เวลาเรียกใช้คอมโพเนนต์ย่อยนี้ เราไม่จำเป็นต้องป้อนค่าพร็อพส์ตัวนี้เสมอไป ซึ่งช่วยรันระบบได้เป็นระเบียบและลดคำสั่งแจ้งเตือนความผิดพลาด (Linter warning)

• **ค. ผิด:** ยังคงสามารถอ้างถึงและใช้งานตัวแปรได้ตามปกติ เพียงแต่ค่าอาจเป็น `undefined` หากไม่มีการส่งมาจากแม่

• **ง. ผิด:** ระบบ TypeScript ทำหน้าที่เพียงตรวจสอบประเภทตัวแปร ไม่สามารถประดิษฐ์สุ่มคำให้เกิดขึ้นเองรันไทม์ได้

ปฏิบัติการท้ายบทเรียน: การสร้างหน้าจอโปรไฟล์และฟอร์มตอบรับข้อมูลผู้ใช้งาน

ปฏิบัติการนี้จะฝึกฝนทักษะการประยุกต์ใช้คอมโพเนนต์พื้นฐานหลักของ React Native และแนวคิดการสร้างคอมโพเนนต์ย่อยที่กำหนดขึ้นเอง (Custom Component) ที่สนับสนุนโดย

กลไก Type-safety ด้วย TypeScript Interface โดยมีเป้าหมายคือการพัฒนาหน้าจอ **ระบบข้อมูลโปรไฟล์พร้อมฟอร์มกรอกข้อมูลแบบโต้ตอบ (Profile Form with Live Preview Card)**

1. วัตถุประสงค์การเรียนรู้

1. สามารถใช้คอมโพเนนต์ `SafeAreaView` และ `ScrollView` เพื่อรองรับการทำงานของหน้าจอที่เนื้อหายาวโดยไม่ทับซ้อนขอบจอ
2. สามารถใช้คอมโพเนนต์ `Image` ในการแสดงผลภาพจากเน็ตเวิร์กได้อย่างถูกวิธี
3. สามารถประยุกต์ใช้ `TextInput` ในการรับข้อมูลรูปแบบต่าง ๆ พร้อมจัดการการทำงานของแป้นพิมพ์ (Keyboard Control)
4. สามารถควบคุมสไตล์ชีทแบบตอบสนองเมื่อผู้ใช้กดสัมผัสด้วย `Pressable` และพารามิเตอร์ `{ pressed }`
5. เข้าใจและลงมือสร้าง Custom Component พร้อมเขียน TypeScript Interface เพื่อกำกับประเภทตัวแปรของ Props ได้อย่างมีหลักการ

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องแก้ไขไฟล์ภายในบทเรียนโดยสร้างโฟลเดอร์สำหรับ Custom Component และเขียนโค้ดประสานงานกันบนหน้าหลัก `app/index.tsx` ดังนี้:

โครงสร้างการจัดแสดงข้อมูลเลย์เอาต์:

1. **พื้นที่ปลอดภัย (`SafeAreaView`):** คลุมรอบนอกสุดเพื่อป้องกันเนื้อหาทับซ้อนช่องบากบน iOS
2. **พื้นที่เลื่อนข้อความ (`ScrollView`):** ครอบเนื้อหาทั้งหมดเพื่อเปิดโอกาสให้เลื่อนสไลด์หน้าจอด้านล่างขึ้นมาได้ยามป้อนข้อมูลผ่านแป้นพิมพ์
3. **การ์ดโปรไฟล์แสดงผล (Live Preview Card):**
   - สร้างเป็น Custom Component ชื่อ `ProfileCard` ไว้ในไฟล์แยกต่างหากคือ `components/ProfileCard.tsx`
   - มีคุณสมบัติ Props ใน TypeScript กำกับ ได้แก่ `avatarUrl: string`, `name: string`, `role: string`, `statusText?: string`, และ `onPressCard?: () => void`
   - แสดงรูปภาพด้วย `Image` (สไตล์ทรงกลมโค้งมน สมมาตร) พร้อมแสดงชื่อ ตำแหน่งงาน และสเตตัสข้อความ
4. **แบบฟอร์มแก้ไขข้อมูลโปรไฟล์ (Profile Form Editor):**

• ช่องพิมพ์รับค่า URL รูปภาพ (TextInput กำหนดคีย์บอร์ดประเภทอีเมล/เว็บและปิดระบบตัวพิมพ์ใหญ่)

• ช่องพิมพ์รับชื่อ (TextInput ทั่วไป)

• ช่องพิมพ์รับตำแหน่งงาน (TextInput ทั่วไป)

• ช่องพิมพ์รับข้อความสเตตัส (TextInput ทั่วไป)

5. **ปุ่มอัปเดตข้อมูล (Pressable Button):**

• จัดสไตล์ด้วย Pressable

• มีเอฟเฟกต์ตอบสนองอย่างรวดเร็ว (ย่อขนาดลงและสีเข้มขึ้น) ขณะกดปุ่มสัมผัส

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: การสร้าง Custom Component พร้อมระบุโครงสร้าง TypeScript Props Interface

สร้างไฟล์ใหม่ในโฟลเดอร์โครงการตามเส้นทาง components/ProfileCard.tsx (หากไม่มีโฟลเดอร์ components ให้สร้างโฟลเดอร์นี้ขึ้นมาใหม่ก่อน) และใส่รหัสด้านล่างนี้ลงไป:

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

// 1. ระบุข้อกำหนดประเภทข้อมูล Props ด้วย TypeScript Interface
export interface ProfileCardProps {
  avatarUrl: string;
  name: string;
  role: string;
  statusText?: string;          // เครื่องหมาย ? แสดงว่าเป็นคุณสมบัติทางเลือก (Optional)
  onPressCard?: () => void;   // เหตุการณ์ฟังก์ชันส่งสัญญาณเมื่อกดการ์ด
}

// 2. นำฟังก์ชันคอมโพเนนต์มาแมปตัวแปร Props
export default function ProfileCard({
  avatarUrl,
  name,
  role,
  statusText = 'ไม่มีข้อความอัปเดตสเตตัสในขณะนี้', // ค่าเริ่มต้นหากไม่มีการส่งมา
  onPressCard
}: ProfileCardProps) {

 // กำหนดรูปภาพสำรองหาก URL ถูกส่งมาเป็นข้อความว่างเปล่า
  const validAvatar = avatarUrl.trim() !== ''
    ? avatarUrl
  : 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?w=150';

  return (
    <Pressable
```

```
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { opacity: pressed ? 0.95 : 1.0, transform: [{ scale: pressed ? 0.99 : 1.0
}] }
    ]}
    >
      {/* แสดงภาพโปรไฟล์จาก URL เครือข่าย */}
      <Image source={{ uri: validAvatar }} style={styles.avatar} />

      <View style={styles.infoContainer}>
        <Text style={styles.nameText}>{name}</Text>
        <Text style={styles.roleText}>{role}</Text>
        <View style={styles.statusBox}>
          <Text style={styles.statusLabelText}>สเตตัสปัจจุบัน:</Text>
          <Text style={styles.statusText}>{statusText}</Text>
        </View>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    flexDirection: 'row',
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 6,
    elevation: 4,
    marginVertical: 12,
  },
  avatar: {
    width: 80,
    height: 80,
    borderRadius: 40, // ครึ่งหนึ่งของขนาดภาพดึงมุมกลมสมบูรณ์
    backgroundColor: '#334155',
  },
  infoContainer: {
    flex: 1,
    marginLeft: 16,
  },
```

```
  nameText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  roleText: {
    fontSize: 14,
    color: '#38bdf8',
    marginTop: 2,
  },
  statusBox: {
    marginTop: 8,
  borderTopWidth: 1,
    borderTopColor: '#334155',
    paddingTop: 6,
  },
  statusLabelText: {
    fontSize: 10,
    color: '#94a3b8',
    textTransform: 'uppercase',
  },
  statusText: {
    fontSize: 12,
    color: '#cbd5e1',
    fontStyle: 'italic',
   marginTop: 2,
  },
});
```

ขั้นตอนที่ 2: วางหน้าจอและเก็บสถานะฟอร์มในคอมโพเนนต์หลัก app/index.tsx

เขียนโครงสร้างการรับข้อมูล และการแสดงตัวอย่างการ์ดร่วมกับแบบฟอร์มการพิมพ์ โดยใช้ State ในการสะท้อนข้อมูลแบบ Real-time:

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import ProfileCard from '../components/ProfileCard'; // เรียกใช้ออบเจกต์ Custom Component

export default function App() {
```

```
  // ประกาศ State เพื่อผูกเข้ากับฟอร์ม TextInput
  const [name, setName] = useState('สมชาย ใจดี');
  const [role, setRole] = useState('Full Stack Developer');
  const [avatar, setAvatar] = useState('https://images.unsplash.com/photo-
1535713875002-d1d0cf377fde?w=150');
  const [status, setStatus] = useState('กำลังเรียนรู้ React Native Expo SDK 54');

  // สเตตสำหรับการบันทึกจริงตอนกดปุ่ม
  const [savedData, setSavedData] = useState({
    name: 'สมชาย ใจดี',
    role: 'Full Stack Developer',
    avatar: 'https://images.unsplash.com/photo-1535713875002-
d1d0cf377fde?w=150',
    status: 'กำลังเรียนรู้ React Native Expo SDK 54'
  });

  const handleUpdate = () => {
    setSavedData({ name, role, avatar, status });
    Alert.alert('อัปเดตสำเร็จ', 'ข้อมูลโปรไฟล์ได้รับการบันทึกบนเครื่องสมบูรณ์');
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>

        <Text style={styles.pageTitle}>ระบบจัดการโปรไฟล์ผู้ใช้</Text>
        <Text style={styles.sectionTitle}>การ์ดแสดงผลปัจจุบัน (Preview)</Text>

        {/* เรียกใช้งาน Custom Component และส่งผ่านข้อมูลผ่านโครงสร้าง Props */}
        <ProfileCard
          name={savedData.name}
          role={savedData.role}
          avatarUrl={savedData.avatar}
          statusText={savedData.status}
          onPressCard={() => Alert.alert('ข้อความสเตตัส', savedData.status)}
    />

        <View style={styles.formContainer}>
          <Text style={styles.sectionTitle}>ฟอร์มกรอกข้อมูลการแก้ไข</Text>

          {/* ช่องกรอกรูปภาพ Avatar URL */}
          <Text style={styles.label}>URL รูปภาพโปรไฟล์</Text>
          <TextInput
            style={styles.input}
            value={avatar}
          onChangeText={setAvatar}
            placeholder="ใส่ URL รูปภาพโปรไฟล์..."
```

```
            keyboardType="url"
            autoCapitalize="none"
            autoCorrect={false}
        />

        {/* ช่องกรอกชื่อ */}
        <Text style={styles.label}>ชื่อ-นามสกุล</Text>
        <TextInput
            style={styles.input}
            value={name}
            onChangeText={setName}
            placeholder="ป้อนชื่อและนามสกุล..."
        />

        {/* ช่องกรอกตำแหน่งงาน */}
        <Text style={styles.label}>ตำแหน่งงาน/บทบาท</Text>
        <TextInput
            style={styles.input}
            value={role}
            onChangeText={setRole}
            placeholder="เช่น UI/UX Designer..."
        />

        {/* ช่องกรอกสเตตัส */}
        <Text style={styles.label}>ข้อความสถานะ (Status Message)</Text>
        <TextInput
            style={[styles.input, styles.textArea]}
            value={status}
            onChangeText={setStatus}
            placeholder="ใส่ข้อความทักทายของคุณ..."
            multiline={true}
            numberOfLines={3}
        />

        {/* ปุ่มบันทึกจัดสไตล์ด้วย Pressable */}
        <Pressable
            onPress={handleUpdate}
            style={({ pressed }) => [
                styles.submitButton,
                {
                    backgroundColor: pressed ? '#0369a1' : '#0284c7',
                    transform: [{ scale: pressed ? 0.98 : 1.0 }]
                }
            ]}
        >
            <Text style={styles.submitButtonText}>อัปเดตโปรไฟล์</Text>
        </Pressable>
```

```
        </View>
      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a', // คลุมธีมสีกึ่งน้ำเงินเข้ม
  },
  scrollContainer: {
    padding: 20,
    paddingBottom: 40,
  },
  pageTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 8,
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: '600',
    color: '#94a3b8',
    marginTop: 12,
    marginBottom: 4,
  },
  formContainer: {
    marginTop: 16,
    backgroundColor: '#1e293b',
    padding: 20,
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
  },
  label: {
    color: '#cbd5e1',
    fontSize: 14,
  fontWeight: '500',
    marginTop: 12,
    marginBottom: 6,
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
```

```
    borderColor: '#334155',
    borderRadius: 8,
    height: 46,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 14,
  },
  textArea: {
    height: 80,
    paddingTop: 10,
    textAlignVertical: 'top', // ป้องกันข้อความเด้งไปกึ่งกลางแนวแกนตั้งในระบบ Android
  },
  submitButton: {
    borderRadius: 8,
    height: 48,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 20,
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ด้านล่างนี้คือไฟล์โค้ดเฉลยฉบับรันได้จริง ประกอบด้วยโค้ดแยกตามโครงสร้างย่อย เพื่อให้ผู้เรียนสามารถประยุกต์ทำปฏิบัติการและนำโครงสร้างนี้ไปรันผลลัพธ์ผ่านหน้าจอจำลองได้ทันที:

4.1 ซอร์สโค้ดไฟล์คอมโพเนนต์: components/ProfileCard.tsx

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

export interface ProfileCardProps {
  avatarUrl: string;
  name: string;
  role: string;
  statusText?: string;
  onPressCard?: () => void;
}

export default function ProfileCard({
  avatarUrl,
  name,
  role,
```

```
  statusText = 'ไม่มีข้อความอัปเดตสถานะ',
  onPressCard
}: ProfileCardProps) {

  const validAvatar = avatarUrl.trim() !== ''
    ? avatarUrl
  : 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?w=150';

  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { opacity: pressed ? 0.95 : 1.0, transform: [{ scale: pressed ? 0.99 : 1.0
}] }
   ]}
    >
      <Image source={{ uri: validAvatar }} style={styles.avatar} />
      <View style={styles.infoContainer}>
        <Text style={styles.nameText}>{name}</Text>
        <Text style={styles.roleText}>{role}</Text>
        <View style={styles.statusBox}>
          <Text style={styles.statusLabelText}>สเตตัสปัจจุบัน:</Text>
          <Text style={styles.statusText}>{statusText}</Text>
        </View>
      </View>
    </Pressable>
 );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    flexDirection: 'row',
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 6,
    elevation: 4,
    marginVertical: 12,
  },
  avatar: {
    width: 80,
```

```
    height: 80,
    borderRadius: 40,
    backgroundColor: '#334155',
  },
  infoContainer: {
    flex: 1,
    marginLeft: 16,
  },
  nameText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  roleText: {
    fontSize: 14,
    color: '#38bdf8',
    marginTop: 2,
  },
  statusBox: {
    marginTop: 8,
    borderTopWidth: 1,
    borderTopColor: '#334155',
    paddingTop: 6,
  },
  statusLabelText: {
    fontSize: 10,
    color: '#94a3b8',
    textTransform: 'uppercase',
  },
  statusText: {
    fontSize: 12,
    color: '#cbd5e1',
    fontStyle: 'italic',
    marginTop: 2,
  },
});
```

4.2 ซอร์สโค้ดหน้าจอแอปหลัก: **app/index.tsx**

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
```

```
  Alert
} from 'react-native';
import ProfileCard from '../components/ProfileCard';

export default function App() {
  const [name, setName] = useState('สมชาย ใจดี');
  const [role, setRole] = useState('Full Stack Developer');
  const [avatar, setAvatar] = useState('https://images.unsplash.com/photo-
1535713875002-d1d0cf377fde?w=150');
  const [status, setStatus] = useState('กำลังเรียนรู้ React Native Expo SDK 54');

  const [savedData, setSavedData] = useState({
    name: 'สมชาย ใจดี',
    role: 'Full Stack Developer',
    avatar: 'https://images.unsplash.com/photo-1535713875002-
d1d0cf377fde?w=150',
    status: 'กำลังเรียนรู้ React Native Expo SDK 54'
  });

  const handleUpdate = () => {
    setSavedData({ name, role, avatar, status });
    Alert.alert('อัปเดตสำเร็จ', 'ข้อมูลโปรไฟล์ได้รับการบันทึกบนเครื่องสมบูรณ์');
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>

        <Text style={styles.pageTitle}>ระบบจัดการโปรไฟล์ผู้ใช้</Text>
        <Text style={styles.sectionTitle}>การ์ดแสดงผลปัจจุบัน (Preview)</Text>

        <ProfileCard
          name={savedData.name}
          role={savedData.role}
          avatarUrl={savedData.avatar}
          statusText={savedData.status}
          onPressCard={() => Alert.alert('ข้อความสเตตัส', savedData.status)}
    />

        <View style={styles.formContainer}>
          <Text style={styles.sectionTitle}>ฟอร์มกรอกข้อมูลการแก้ไข</Text>

          <Text style={styles.label}>URL รูปภาพโปรไฟล์</Text>
          <TextInput
            style={styles.input}
            value={avatar}
            onChangeText={setAvatar}
            placeholder="ใส่ URL รูปภาพโปรไฟล์..."
```

```
            keyboardType="url"
            autoCapitalize="none"
            autoCorrect={false}
        />

        <Text style={styles.label}>ชื่อ-นามสกุล</Text>
        <TextInput
            style={styles.input}
            value={name}
            onChangeText={setName}
            placeholder="ป้อนชื่อและนามสกุล..."
        />

        <Text style={styles.label}>ตำแหน่งงาน/บทบาท</Text>
        <TextInput
            style={styles.input}
            value={role}
            onChangeText={setRole}
            placeholder="เช่น UI/UX Designer..."
        />

        <Text style={styles.label}>ข้อความสถานะ (Status Message)</Text>
        <TextInput
            style={[styles.input, styles.textArea]}
            value={status}
            onChangeText={setStatus}
            placeholder="ใส่ข้อความทักทายของคุณ..."
            multiline={true}
            numberOfLines={3}
        />

        <Pressable
            onPress={handleUpdate}
            style={({ pressed }) => [
              styles.submitButton,
              {
                backgroundColor: pressed ? '#0369a1' : '#0284c7',
                transform: [{ scale: pressed ? 0.98 : 1.0 }]
              }
        ]}
        >
            <Text style={styles.submitButtonText}>อัปเดตโปรไฟล์</Text>
        </Pressable>

      </View>
    </ScrollView>
  </SafeAreaView>
);
}
```

```
const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  scrollContainer: {
    padding: 20,
    paddingBottom: 40,
  },
  pageTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 8,
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: '600',
    color: '#94a3b8',
    marginTop: 12,
    marginBottom: 4,
  },
  formContainer: {
    marginTop: 16,
    backgroundColor: '#1e293b',
    padding: 20,
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
  },
  label: {
    color: '#cbd5e1',
    fontSize: 14,
    fontWeight: '500',
    marginTop: 12,
    marginBottom: 6,
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 46,
    paddingHorizontal: 12,
```

```
    color: '#ffffff',
    fontSize: 14,
  },
  textArea: {
    height: 80,
    paddingTop: 10,
    textAlignVertical: 'top',
  },
  submitButton: {
    borderRadius: 8,
    height: 48,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 20,
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

React Native. (2024). *Core Components and APIs*. จาก https://reactnative.dev/docs/intro-react-native-components

React Native. (2024). *Image*. จาก https://reactnative.dev/docs/image

React Native. (2024). *Pressable*. จาก https://reactnative.dev/docs/pressable

React Native. (2024). *Text*. จาก https://reactnative.dev/docs/text

React Native. (2024). *TextInput*. จาก https://reactnative.dev/docs/textinput

React Native. (2024). *View*. จาก https://reactnative.dev/docs/view